Helios — Execution Guide

The one document to read before you build. Architecture is frozen. This optimizes for *shipping*, not planning. · 2026-06-03

> **The single most important fact about this repo:** the architecture and documentation are ~100% done; the *code* is ~2% done. Your enemy is not under-specification — it's **documentation overload and analysis paralysis**. Read the four CRITICAL docs, then write code. Everything else is reference.

PART 1 — Reading Plan (prevent overload)

The minimum to start building (≈ 2.5 hrs, read in this order): 1. `CLAUDE.md` (root) — 20 min 2. `docs/planning/LEAN_SCOPE.md` — 15 min 3. `docs/architecture/DBT_GUIDE.md` §1–§5 — 40 min 4. `docs/architecture/DATA_MODEL.md` §5 (session model) — 15 min 5. `docs/planning/IMPLEMENTATION_PLAYBOOK.md` M0–M1 only — 20 min

That's it. Open everything else **only when a milestone needs it.** Do **not** read the Bible or the Interview Guide cover-to-cover — that is the trap.

Document	Priority	Read complete?	When	Time	Why it exists / what to learn
`CLAUDE.md`	**Critical**	Complete	Now, first	20m	Your operating anchor: canonical names, grounding rules G1–G5, dbt conventions, the keystones. Auto-loads into every Claude Code session. Learn: the non-negotiable rules + vocabulary.
`docs/planning/LEAN_SCOPE.md`	**Critical**	Complete	Now	15m	The ruthless cut: MVP / v1 / v2 and what to kill. Learn: *what you are actually building* (not the full design).
`docs/architecture/DBT_GUIDE.md`	**Critical**	§1–§5 complete; §6–§9 reference	Week 1–2	40m	The literal, copy-usable dbt code (config, sources, staging, the keystones, marts). Learn: exactly what to build for M0–M4.
`docs/architecture/DATA_MODEL.md`	**Critical (§5)**	§5 complete; rest reference	Week 1 (M3)	15m	The session model — sessionization, `traffic_source` fallback, monotonic `reached_*`. Learn: the keystone that fails silently.
`docs/planning/IMPLEMENTATION_PLAYBOOK.md`	**Critical**	Reference, milestone-by-milestone	Each milestone	per-M	The build manual: objective / files / commands / tests / success / mistakes per milestone. Use as a checklist; don't pre-read it all.
`models/semantic/semantic_layer.yaml`	**Critical**	Reference (per metric)	When building semantic-mcp / metrics	—	The governed registry (47 metrics). Learn: the field schema; look up a metric's SQL when you need it. Never read end-to-end.
`docs/strategy/RED_TEAM_REVIEW.md`	**Important**	Complete, once	Now (before building)	20m	The honest critique. Learn the traps to avoid: don't claim "autonomous"/"diagnose why", don't build the fleet, keep the eval honest. Shapes your framing.
`docs/strategy/CLAUDE_CODE_WORKFLOW.md`	**Important**	Reference, per milestone	While coding with Claude	per-M	How to drive Claude Code without hallucination: what to attach, prompts, review rubric. Use the per-milestone section you're on.
`docs/architecture/MCP_ARCHITECTURE.md`	**Important**	§6 + §9 complete; rest reference	Week 3 (M6)	25m	The MCP server you'll build. Learn: the guardrail chain + the §9 Python skeletons (copy them).
`docs/planning/DEVELOPMENT_PLAN.md`	**Important**	Reference (the tracker)	Throughout	—	Milestone tracker + exit gates + risks. Use the §12 tracker to know where you are.
`docs/architecture/METRIC_GOVERNANCE_GUIDE.md`	**Reference**	Reference	M5 (semantic layer)	—	How the registry is validated/versioned + the field-name→resolver mapping. Read §3 + §6 when building `validate_semantic.py` /semantic-mcp.
`docs/architecture/METRIC_DEPENDENCY_GRAPH.md`	**Reference**	Reference	When coding decomposition/RCA	—	Metric dependency trees + identities (RPS = conv × AOV). Consult for the decomposition logic.
`docs/architecture/AGENT_ARCHITECTURE.md`	**Reference**	§6.7 only for lean; rest = v2	Week 3 (M7 brief)	—	The 7-agent design. For the LEAN build you need **only §6.7 (Narrator brief) + the Finding shape**. The rest is v2 — ignore for now.
`docs/architecture/HELIOS_PROJECT_BIBLE.md`	**Reference**	NEVER read whole (233 KB)	Look up specific § only	—	The master spec. Everything is distilled into CLAUDE.md. Jump to a numbered § only when a reference doesn't answer your question.
`docs/planning/DEPENDENCY_MAP.md`	**Reference**	Reference	When unsure of build order	—	The critical path + what-blocks-what. Consult if you're unsure what to build next.

Document	Priority	Read complete?	When	Time	Why it exists / what to learn
`README.md`	**Ignore for now**	Skim once	—	5m	An external-context manifest. Skim; don't study.
`eval/scenarios/` (BENCHMARK)	**Reference**	Reference	Week 4 (M10)	—	The 50 labeled scenarios. For the lean eval you'll **reuse 6–10** of these as the format; don't regenerate.
`docs/strategy/INTERVIEW_GUIDE.md`	**Ignore for now**	—	After you've built v1	—	Job-hunt prep, not build material. Open it when you're interviewing.
Migration/Reconciliation/Restructure reports	**Ignore**	—	Never (housekeeping)	—	Records of the repo reorg. No build value.

PART 2 — Implementation Readiness Review *(principal-engineer view)*

✅ **Complete (do not re-do):** - All **design + documentation** (Bible, data model, dbt code, MCP/agent specs, governance, build plan, the red-team). - The **semantic registry** `models/semantic/semantic_layer.yaml` — 47 metrics / 19 dims, referential-integrity validated (0 dangling refs). A real, finished asset. - The **benchmark** `eval/scenarios/` — 50 labeled scenarios, validated. A real, finished asset. - The **repo structure**, conventions, and canonical vocabulary (post-restructure, post-reconciliation, internally consistent).

❌ **Missing (this is the entire job):** - **All running code.** No dbt project (`dbt_project.yml` / `profiles.yml` / `packages.yml`), no models, no macros, no GCP/BigQuery connection. - No MCP servers, no agents, no LLM brief, no eval harness code, no CI. - Specced-but-not-written helpers: `scripts/validate_semantic.py` ; the **semantic-mcp field-name adapter** (`metric_name→name` , `sql_definition→sql` , `aggregation_method→agg` , `dimensions_supported→dimensions`) the v2 registry needs at load. - Zero tests have ever run; the 85%/<5min/0-hallucination numbers are **targets, not measurements.**

🗑️ **Ignore (per LEAN_SCOPE + red-team):** the 7-agent fleet, the Critic loop, memory/vector store, scheduler/autonomy, `experiment-mcp` + power analysis, cohorts/retention/RFM/forecasting, 4 of the 5 MCP servers, the full 50-scenario CI gate, multi-tenant/warehouse-agnostic, `backend/` + `frontend/` .

🔒 **Never touch again (frozen):** the semantic-layer **structure** (extend via governance, never rewrite); the **canonical names/conventions** in CLAUDE.md; the **decomposition identity** (math is settled); the **funnel/grain/** `session_key` **/** `reached_*` definitions; the **10 channel groups**; `docs/archive/**` . Changing any of these invalidates everything downstream.

🔗 **Assumptions already locked in (don't relitigate):** GA4 Google-Merch-Store dataset (static · ~3 months · obfuscated · `user_id` null → cookie-grain); stack = BigQuery + dbt + Python + MCP + Claude; **mix-vs-rate decomposition is the centerpiece**; **governed-SQL grounding** (LLM never writes SQL); data is **observational** (you *design/size* experiments, never run live A/Bs); the honest reframe — Helios diagnoses **WHERE** (which segment, mix vs rate), not **WHY**; it is a **diagnosis assistant**, not an autonomous always-on product.

One 2-minute decision before Week 1: run the dbt project **from the repo root** (`dbt_project.yml` at root, `models/{staging,intermediate,marts}/` alongside the existing `models/semantic/`), matching `DBT_GUIDE.md §1` . Leave the empty `dbt/` dir unused (or delete it) — do **not** move the validated registry.

PART 3 — MVP / v1 / v2 *(ruthless)*

Helios MVP — "governed mix-vs-rate diagnosis on real GA4 data. No LLM." dbt spine (staging → `int_ga4__sessionized` → `int_ga4__funnel_steps` → `fct_funnel` , `fct_daily_funnel` , `fct_orders` , + `dim_date` / `dim_channels`), tested (monotonicity, reconcile-to-the-cent) · a ~12-metric subset of the registry · `decompose_change` in Python (golden-tested) · a `diagnose.py` that finds the biggest WoW funnel move, decomposes it (mix vs rate by device/channel), prices it, and prints a **templated** brief. **This alone is a strong portfolio piece.**

Helios v1 — "+ one MCP server + one grounded LLM brief + an honest eval." *(this is what you ship)* One MCP server (`semantic-mcp` *or* `stats-mcp`) wired to Claude Code → the LLM composes governed metrics / calls deterministic math, never writes SQL (0 hallucinated columns) · the **Decision Brief** written by Claude from the deterministic decomposition + dollar figure · an honest **6–10 scenario** benchmark vs the naive "largest-segment-delta" baseline, framed as controlled-attribution accuracy (not causal).

Helios v2 — postpone all of this: the other 4 MCP servers · the full 7-agent FSM + Critic loop · memory/vector store · scheduler/autonomy · `experiment-mcp` /power analysis/experiment design · cohorts/retention/RFM/forecasting · the full 50-scenario CI gate · multi-tenant/warehouse-agnostic adapters · the frontend. **And permanently drop:** "diagnose WHY", "autonomous", causal claims, the circular self-graded benchmark framing.

Postpone list, one line: *autonomy, the agent fleet, memory, experiment execution, retention/LTV, the UI — none of it is demonstrable or necessary on a frozen 3-month dataset with one developer.*

PART 4 — 4-Week Build Plan *(solo · 20–25 hr/wk · ~90 hrs total)*

Week 1 — Foundation + Data Spine (M0–M4) → the MVP's data half

- **Objectives:** GCP/BigQuery + dbt connected; the governed marts built and tested.
- **Deliverables:** working `dbt build`; `fct_funnel` / `fct_daily_funnel` / `fct_orders` + `dim_date` / `dim_channels`; passing monotonicity + revenue-reconciliation tests.
- **Files to create:** `requirements.txt`, `dbt_project.yml`, `profiles.yml`, `packages.yml`; `models/staging/src_ga4.yml`, `stg_ga4__events.sql`, `stg_ga4__event_params.sql`; `macros/{get_event_param,sessionize,channel_group,test_revenue_reconciles}.sql`; `seeds/channel_group_mapping.csv`; `models/intermediate/int_ga4__sessionized.sql`, `int_ga4__funnel_steps.sql`; `models/marts/ ...` (the marts) + `schema.yml` tests; `tests/assert_funnel_monotonicity.sql`. **Copy from `DBT_GUIDE.md` §1–§5; cross-check grains in `DATA_MODEL.md` §5/§8.**
- **Commands:** `python -m venv .venv` → `pip install -r requirements.txt`; `gcloud auth application-default login` (+ least-privilege SA); `dbt deps`; `dbt debug`; `dbt build`; `dbt build --select +fct_funnel`; `dbt test`.
- **Learn:** dbt (models, refs, tests, incremental), BigQuery GA4 nested schema (`UNNEST(event_params)`), `_TABLE_SUFFIX` pruning.
- **Success:** `dbt build` green; `revenue_reconciles` to the cent; `sessions ≥ reached_view_item ≥ … ≥ reached_purchase`; `dim_channels` = exactly 10 groups.
- **Risks:** sessionization correctness (it fails *silently* — write the golden tests first); BigQuery cost (cap `maximum_bytes_billed`, prune shards); GA4 struct paths (`device.web_info.browser`, not `device.browser`).

Week 2 — Semantic Layer Live + Decomposition → MVP complete (M5 + the engine)

- **Objectives:** registry compiles against the real marts; the mix-vs-rate engine works and is golden-tested; the no-LLM diagnosis runs end-to-end.
- **Deliverables:** `validate_semantic.py` → 0 dangling refs; `decompose_change` passing its golden test; **MVP demo** (`diagnose.py` prints a real, priced, mix-vs-rate diagnosis of the GMS funnel).
- **Files to create:** `scripts/validate_semantic.py`; `models/semantic/metrics__schema.yml`; `helios/stats/decompose.py` (+ a two-proportion significance test); `tests/` golden unit test for the decomposition; `diagnose.py` (templated brief).
- **Commands:** `python scripts/validate_semantic.py`; `pytest`; `python -m diagnose` (or `python diagnose.py`).
- **Learn:** scipy/statsmodels (two-proportion test), the registry field-name schema, the decomposition algebra (`mix/rate/interaction`).
- **Success:** registry validator prints PASS (0 dangling); `decompose_change` reproduces the golden example (`mix=-0.0018, rate=0, interaction=0`); one anomaly diagnosed end-to-end with a dollar figure.
- **Risks:** sparse/zero-denominator segments in the decomposition (guard them); the registry uses descriptive field names — the validator must read `metric_name` / `sql_definition`, not `name` / `sql`.

Week 3 — One MCP Server + Grounded LLM Brief (M6 partial + M7 lean) → v1 core

- **Objectives:** the LLM reaches data/math only through a governed tool; the Decision Brief is LLM-written but numbers come from tools.
- **Deliverables:** one working MCP server registered with Claude Code; a grounded Decision Brief from the deterministic decomposition.
- **Files to create:** `helios/mcp/base.py`, `helios/mcp/semantic.py` **or** `helios/mcp/stats.py` (copy the `MCP_ARCHITECTURE.md §9` skeleton); `mcp_servers.yaml` (registry → `models/semantic/semantic_layer.yaml`); the brief step (Narrator-as-one-call, `AGENT_ARCHITECTURE.md §6.7`).
- **Commands:** `python -m helios.mcp.semantic`; `claude mcp add ...` (register); contract tests `pytest helios/mcp/tests`; run the brief.
- **Learn:** MCP (FastMCP, JSON-RPC, stdio), the Claude Agent SDK / Claude Code MCP client, the guardrail chain (`build_query → dry_run → run_query`).
- **Success:** `build_query→dry_run→run_query→reconcile` round-trips (or `decompose_change` via the stats server); an unknown metric is a hard error; every number in the brief traces to a tool output (0 hallucinated columns).
- **Risks:** MCP learning curve; **Claude Pro budget** (build/test the deterministic parts first; spend the LLM budget only on the brief); the field-name adapter at registry load.

Week 4 — Honest Eval + Polish + Writeup (M10 lean) → v1 shipped

- **Objectives:** prove the engine beats a naive baseline on a small labeled benchmark; package it.
- **Deliverables:** `injector.py` + `scorer.py` + 6–10 scenarios + naive baseline; a recorded accuracy number with honest caveats; a one-command run + a short case study.
- **Files to create:** `helios/eval/injector.py`, `helios/eval/scorer.py`, reuse 6–10 from `eval/scenarios/`, `eval/baselines.md`; `README` case-study section; `run.py` (one-command pipeline).
- **Commands:** `python -m helios.eval` (smoke); record results; `python run.py`.
- **Learn:** the injection mechanism (rate vs volume perturbation), scoring (top-1 accuracy, decomposition error), honest evaluation framing.
- **Success:** Helios beats the naive baseline on the small benchmark; you can state plainly what it proves (controlled attribution) and doesn't (causal); the whole loop runs from one command.
- **Risks: over-claiming** (the eval proves attribution accuracy, not causation — say so); time pressure (cut scenarios before cutting the honesty).

PART 5 — Your First Implementation Task *(ignore weeks 2–4)*

Task: Stand up the toolchain and prove dbt can reach the GA4 data. Create `requirements.txt` + the three dbt config files, authenticate to BigQuery, and get `dbt debug` green plus one bounded smoke query returning rows. **(This is milestone M0 — nothing else.)**

- **Why it's first:** every other task is untestable until dbt talks to BigQuery. It's the universal unblocker, it's low-risk, and it surfaces environment problems (auth, dataset location = US, billing) *before* you've written a single model. Writing a model first is the classic mistake — you can't run it.
- **Files involved:** `requirements.txt`, `dbt_project.yml`, `profiles.yml`, `packages.yml` — **copy verbatim from** `DBT_GUIDE.md §1` (do not invent values; pin `location: US`, `maximum_bytes_billed: 5368709120`).
- **Commands:** `bash python -m venv .venv && .venv\Scripts\activate # POSIX: source .venv/bin/activate pip install -r requirements.txt gcloud auth application-default login dbt deps dbt debug # smoke query (bounded — confirms data + cost): bq query --use_legacy_sql=false --maximum_bytes_billed=2000000000 \ "SELECT COUNT(DISTINCT CONCAT(user_pseudo_id, CAST((SELECT value.int_value FROM UNNEST(event_params) WHERE key='ga_session_id') AS STRING))) AS sessions \ FROM \`bigquery-public-data.ga4_obfuscated_sample_ecommerce.events_*\` WHERE _TABLE_SUFFIX BETWEEN '20210101' AND '20210107'"`
- **Expected output:** `dbt debug` reports all checks **OK** (connection, auth, project, dataset); the smoke query returns a sessions count for that week (a few thousand) and scans well under the cap.
- **Definition of done:** `dbt debug` passes every check **and** the bounded `events_*` query returns rows under budget. Commit the four config files. *Then* — and only then — start Week 1's macros and staging.

PART 6 — Project Success Probability

Dimension	Estimate	Note
Architecture completion	**~95%**	Complete and frozen — arguably *over*-complete (the red-team's "over-engineered" finding). Nothing more to design.
Documentation completion	**~98%**	Comprehensive, reconciled, internally consistent. Only inline code stubs (e.g. `validate_semantic.py`) remain, and they're specced.
Implementation completion	**~2%**	Only the registry YAML + benchmark scenarios exist as assets. No running code, no dbt, no servers, no tests ever run.
MVP build completion	**~5%**	Spec & design readiness ~95%; *built* ~5%. The MVP's code largely exists in `DBT_GUIDE.md` waiting to be wired — so velocity should be high once you start.

Top 5 levers (in order) to maximize the probability of actually shipping: 1. **Start coding this week — cap your reading.** The #1 failure mode is documentation overload; you have 16 docs and 0 code. Read the 5 CRITICAL items (~2.5 hrs), then do Part 5. Do not re-read. 2. **Connect dbt→BigQuery first (M0).** It unblocks everything and exposes environment issues early. 3. **Write the keystone golden tests *before* the keystone code** (sessionization, `reached_*` monotonicity, `decompose_change`). They fail silently — this is where correctness is won or lost; a wrong keystone poisons every number downstream. 4. **Hold the LEAN line — refuse scope creep.** Resist building the 7-agent fleet, memory, autonomy, or the 4 extra MCP servers. Ship the spine + 1 server + 1 brief + honest eval. The full design is a 6–10 week project; v1 is a 4-week project. 5. **Adopt the red-team's honest reframe.** Build "a governed mix-vs-rate diagnosis assistant with an honest benchmark," not "an autonomous engine that diagnoses why." It's the version you can actually build *and* defend — and it's stronger in an interview than any inflated claim.

> Net: this is a **build-velocity problem, not a design problem.** The plans are done. Open `CLAUDE.md`, then do Part 5 today.